

Wireshark Analysis Assignment

1. What is Wireshark and for what purpose is it used?

Wireshark is a free, open-source network protocol analyzer (also commonly called a packet analyzer or network sniffer). It is widely regarded as the world's most popular tool of its kind and runs on Windows, macOS, Linux, and other platforms.

It works by capturing data packets traveling over a network interface (Ethernet, Wi-Fi, etc.) in real time or from saved capture files, then dissecting and displaying them in a human-readable format. Wireshark understands hundreds of network protocols and shows detailed information about each packet's layers (from physical/link layer up to application layer).

Main purposes and common uses:

- Troubleshooting network problems (slow connections, connection drops, high latency, no response from servers, etc.)
- Diagnosing application communication issues (why an app can't connect, strange errors, protocol misunderstandings)
- Learning and understanding network protocols (great for students, developers, and engineers)
- Verifying network application behavior during development/QA
- Detecting security issues (unencrypted sensitive data, suspicious/malicious traffic patterns, malware communication, unauthorized devices)
- Network performance analysis and bottleneck identification
- Forensics and incident response (analyzing what happened during a security event)

In short: Wireshark lets you "see" exactly what is happening on the network at the packet level, something that is usually hidden from normal applications and users

2. What is packet sniffing?

Packet sniffing (also called packet capture, network sniffing, or protocol analysis) is the practice of intercepting, capturing, and analyzing data packets as they travel across a computer network. Data sent over networks (web browsing, email, file transfers, video calls, gaming, etc.) is broken into small chunks called packets. Each packet contains headers (with source/destination IP, ports, protocol type, etc.) and payload (actual data).

A **packet sniffer** is hardware or software that:

- Puts the network interface into **promiscuous mode** (or monitor mode for Wi-Fi) → captures **all** packets visible on the network segment, not just those addressed to your own device
- Records these packets for real-time viewing or saving to a file (most often .pcap or .pcapng format)
- Allows later analysis of the captured traffic

Legitimate / good uses of packet sniffing:

- Network troubleshooting and diagnostics

- Performance monitoring and bandwidth usage analysis
- Security monitoring (detecting intrusions, data leaks, command & control traffic)
- Debugging applications and protocols
- Compliance auditing and forensics
- Educational purposes (learning how networks really work)

Malicious / bad uses (what most people think of when hearing "sniffing"):

- Stealing unencrypted credentials (usernames, passwords, cookies, session tokens)
- Eavesdropping on conversations (email, chat, VoIP if not encrypted)
- Capturing sensitive data in transit (credit card numbers, personal info)
- Mapping network topology or discovering devices/services

Modern networks mitigate many sniffing risks through:

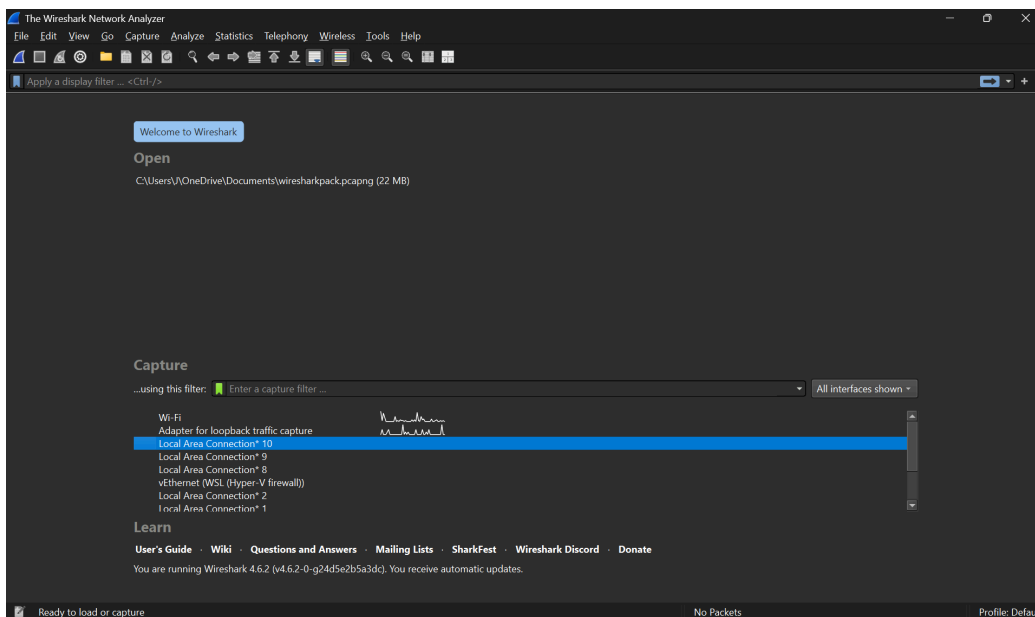
- Encryption (HTTPS/TLS, VPNs, SSH, encrypted VoIP)
- Network segmentation
- Switches instead of old hubs (so only your own traffic is easily visible)

3. Analyzing packets

1. Open the wireshark application.

You will see a screen showing **available network interfaces** such as:

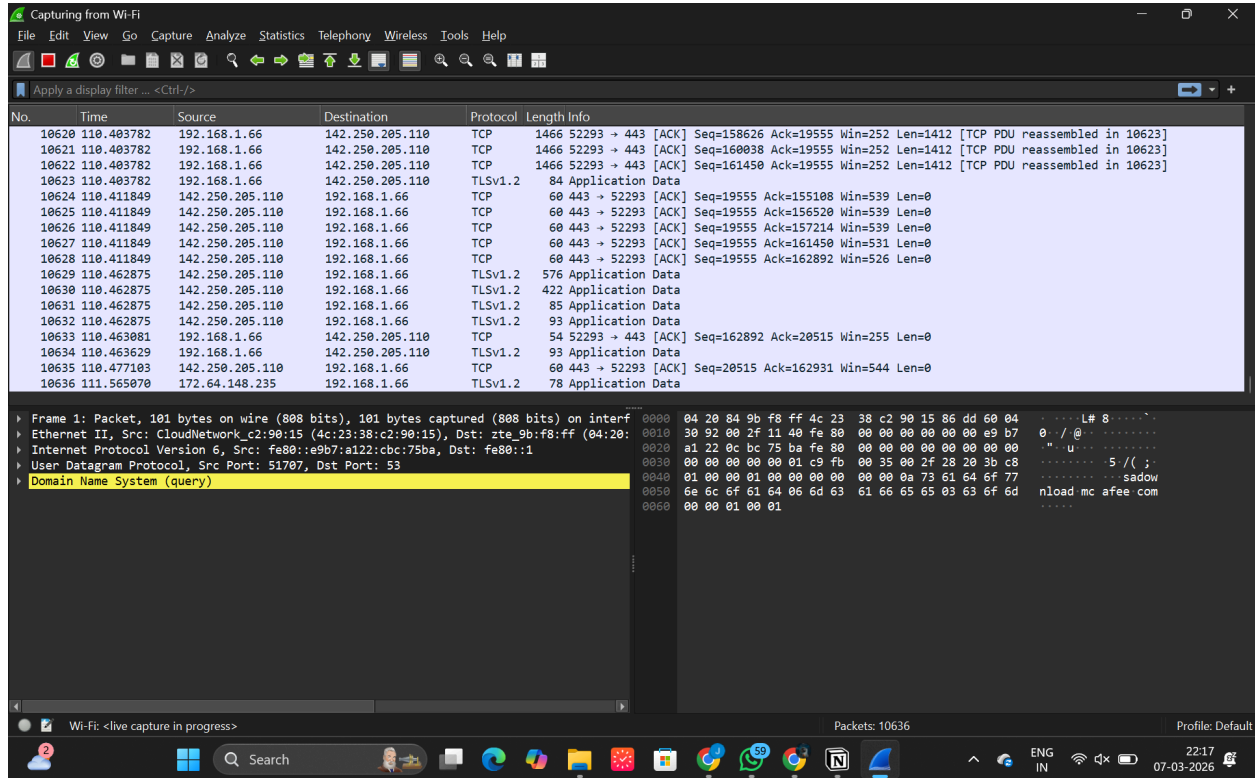
- Wi-Fi
- Ethernet



2. Select the Active Network Interface

- Look for the interface that has moving traffic graphs.
- Usually it is Wi-Fi
- Double-click Wi-Fi.

Wireshark will now start capturing packets automatically.



3. Observe the Packet Capture Window

After capture starts, you will see packets appearing in a list.

Columns include:

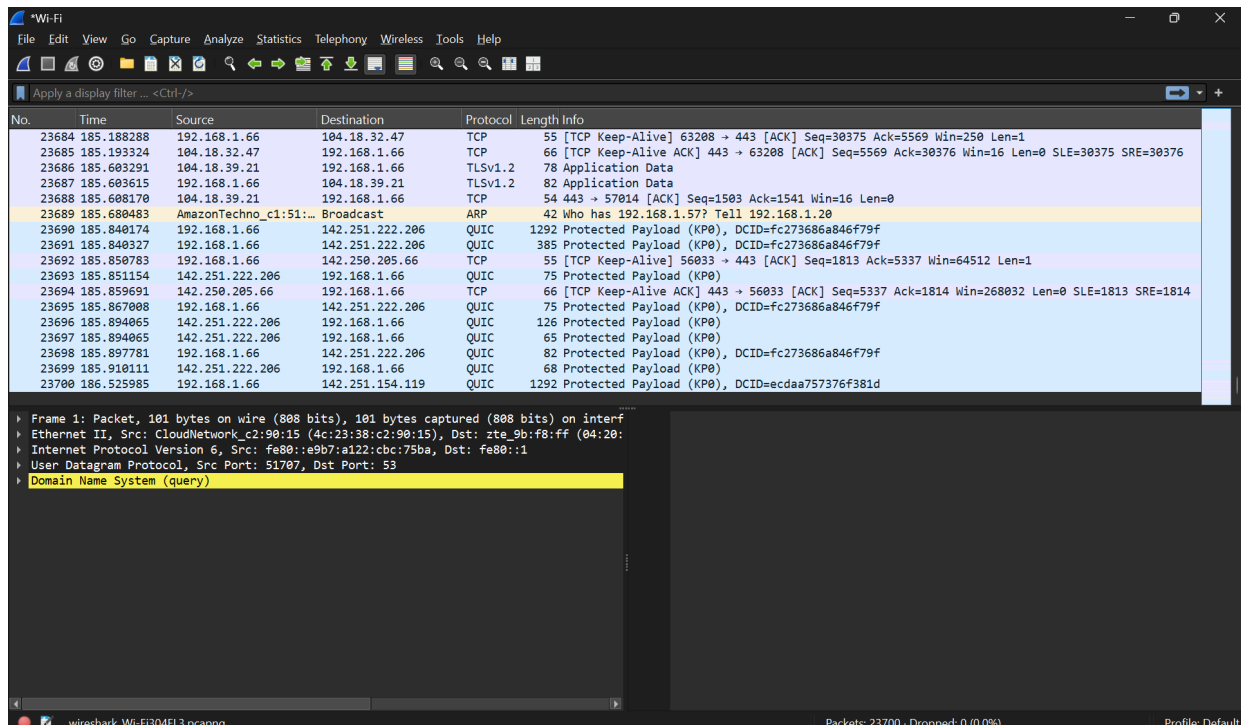
- **No.** – Packet number
- **Time** – Time when packet was captured
- **Source** – Sender IP address
- **Destination** – Receiver IP address
- **Protocol** – Type of protocol (TCP, DNS, TLS, etc.)
- **Length** – Packet size
- **Info** – Additional packet details.

Packet Details (Middle Section)

This shows the **structure of the selected packet**.

screenshot shows:

- Ethernet II
- Internet Protocol Version 6
- UDP
- **Domain Name System (query)**

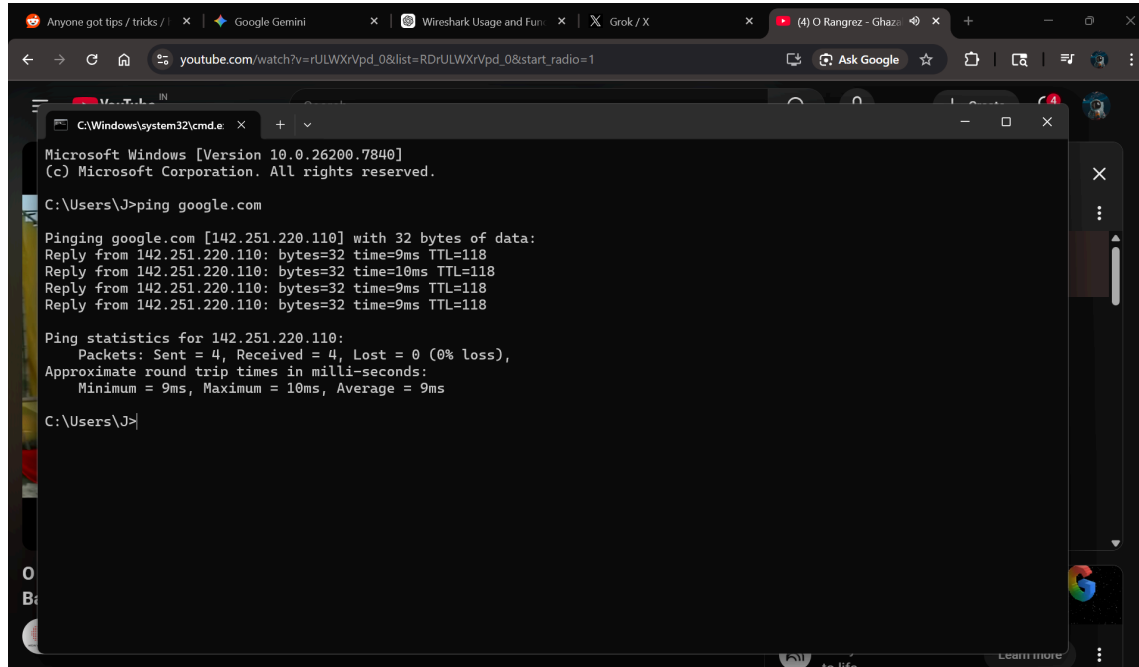


4. Generate Network Traffic

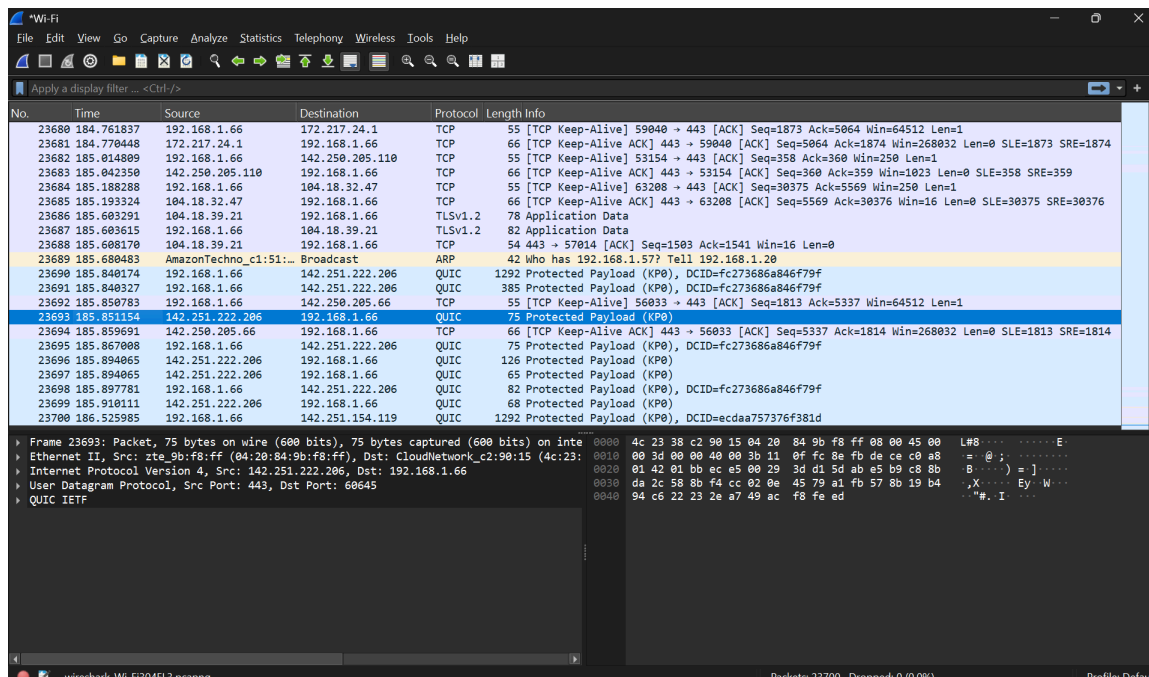
To capture meaningful packets:

- Open Google Chrome / any browser.
- Visit a few websites like:
 - google.com
 - youtube.com
- Play a video.

This creates network traffic, which Wireshark captures. Ping this activity in cmd.



5. Stop packet capture, by clicking the red square button. Packet capture will stop and the captured packets will remain visible for analysis.

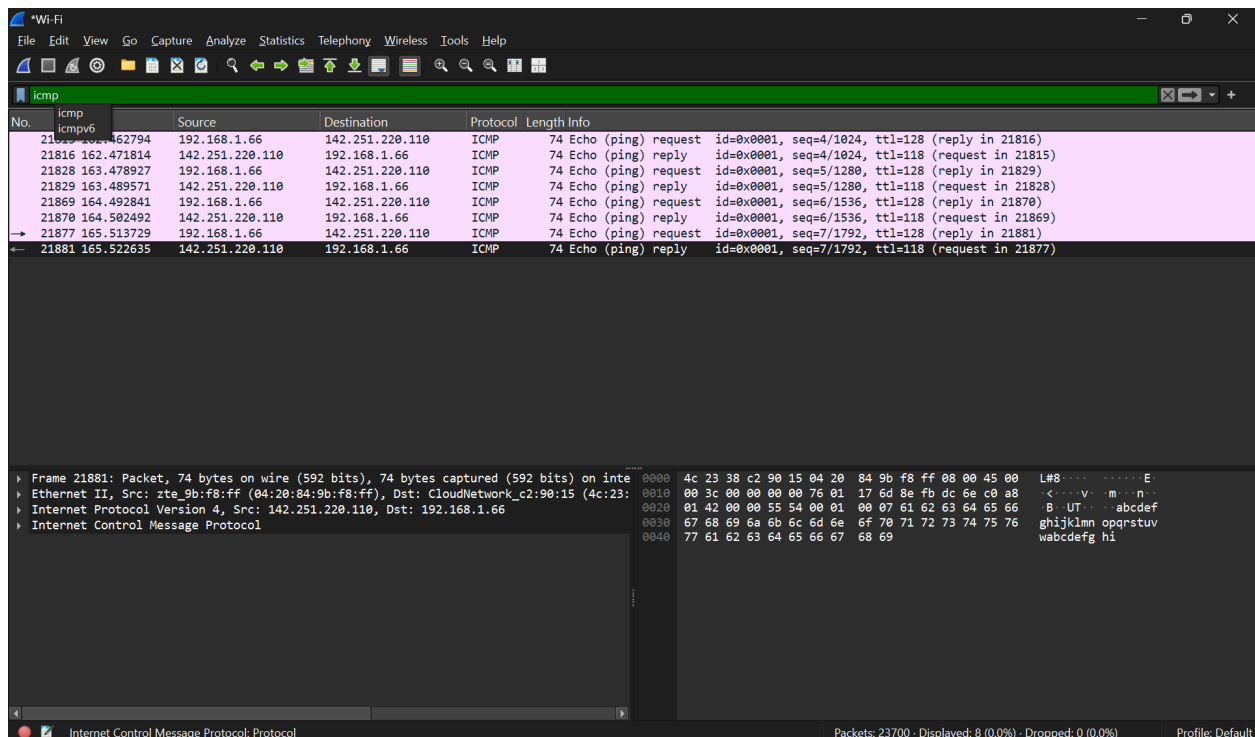


This is the **protocol stack of the packet**.

4. Packet filtering using wireshark.

Step1. Go to the **filter bar at the top**.

Type **icmp**.



These packets came from the command:

->>ping google.com

Now Wireshark will show **only ping packets**.

Step2. Filter DNS packets.

This shows **DNS packet analysis**.

No.	dns	Source	Destination	Protocol	Length	Info
21432	150.360313	fe80::e9b7:a122:cbc...	fe80::1	DNS	119	Standard query 0xbacf A stayfocus.st-panel-api.com
21437	150.373163	fe80::1	fe80::e9b7:a122:cbc...	DNS	152	Standard query response 0xbacf A stayfocus.st-panel-api.com A 34.226.42.245 A 98.89.123.79
21545	154.329176	fe80::e9b7:a122:cbc...	fe80::1	DNS	204	Standard query response 0x53ce HTTPS stayfocus.st-panel-api.com SOA ns-1731.awsdns-24.co.uk
21547	154.329292	fe80::e9b7:a122:cbc...	fe80::1	DNS	112	Standard query 0x2917 A beacons.gcp.gvt2.com
21556	154.417580	fe80::1	fe80::e9b7:a122:cbc...	DNS	159	Standard query response 0x2917 A beacons.gcp.gvt2.com CNAME beacons-handoff.gcp.gvt2.com A ...
21561	154.434118	fe80::1	fe80::e9b7:a122:cbc...	DNS	200	Standard query response 0x5180 HTTPS beacons.gcp.gvt2.com CNAME beacons-handoff.gcp.gvt2.co...
21770	162.265108	fe80::e9b7:a122:cbc...	fe80::1	DNS	150	Standard query 0xbf5f HTTPS 1-prod-dynamite-prod-05-us-signaler-pa.clients6.google.com
21772	162.265256	fe80::e9b7:a122:cbc...	fe80::1	DNS	150	Standard query 0xd4bf A 1-prod-dynamite-prod-05-us-signaler-pa.clients6.google.com
21780	162.373935	fe80::1	fe80::e9b7:a122:cbc...	DNS	167	Standard query response 0xd4bf A 1-prod-dynamite-prod-05-us-signaler-pa.clients6.google.com...
21790	162.404799	fe80::1	fe80::e9b7:a122:cbc...	DNS	201	Standard query response 0xbf5f HTTPS 1-prod-dynamite-prod-05-us-signaler-pa.clients6.google...
21808	162.442930	fe80::e9b7:a122:cbc...	fe80::1	DNS	90	Standard query 0x6509 A google.com
21813	162.451720	fe80::1	fe80::e9b7:a122:cbc...	DNS	106	Standard query response 0x6509 A google.com A 142.251.228.110
22132	175.853120	fe80::e9b7:a122:cbc...	fe80::1	DNS	129	Standard query 0xf958 HTTPS rr1---sn-b5up3cn-cagl.googlevideo.com
22134	175.853367	fe80::e9b7:a122:cbc...	fe80::1	DNS	129	Standard query 0xcfe6 A rr1---sn-b5up3cn-cagl.googlevideo.com
22142	175.926405	fe80::1	fe80::e9b7:a122:cbc...	DNS	180	Standard query response 0xcfe6 A rr1---sn-b5up3cn-cagl.googlevideo.com CNAME rr1.sn-b5up3cn...
22147	175.942124	fe80::1	fe80::e9b7:a122:cbc...	DNS	221	Standard query response 0xf958 HTTPS rr1---sn-b5up3cn-cagl.googlevideo.com CNAME rr1.sn-b5u...
22168	175.952439	fe80::e9b7:a122:cbc...	fe80::1	DNS	106	Standard query 0x81c2 A www.google.com
22170	175.952588	fe80::e9b7:a122:cbc...	fe80::1	DNS	106	Standard query 0xeb58 HTTPS www.google.com
23090	176.034649	fe80::1	fe80::e9b7:a122:cbc...	DNS	132	Standard query response 0xeb58 HTTPS www.google.com HTTPS
23095	176.049806	fe80::1	fe80::e9b7:a122:cbc...	DNS	235	Standard query response 0x81c2 A www.google.com A 142.251.150.119 A 142.251.153.119 A 142.2...

Frame 21813: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on in
 Ethernet II, Src: zte_9b:f8:ff (04:20:84:9b:f8:ff), Dst: CloudNetwork_c2:90:15 (4c:23:00:00:00:00)
 Internet Protocol Version 6, Src: fe80::e9b7:a122:cbc:75ba
 User Datagram Protocol, Src Port: 53, Dst Port: 63249
 Domain Name System (response)

DNS

You will see packets showing **domain name resolution** like:

google.com → IP address

Step 3. Filter HTTPS traffic packets.

Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tls

No.	tls	Time	Source	Destination	Protocol	Length	Info
21745	160.812097	172.64.155.209	192.168.1.66	TLSv1.2	85	Application Data	
21748	162.249476	208.103.161.1	192.168.1.66	TLSv1.2	108	Application Data	
21749	162.249476	208.103.161.1	192.168.1.66	TLSv1.2	108	Application Data	
21750	162.249476	208.103.161.1	192.168.1.66	TLSv1.2	108	Application Data	
21751	162.249476	208.103.161.1	192.168.1.66	TLSv1.2	108	Application Data	
21754	162.252298	192.168.1.66	208.103.161.1	TLSv1.2	112	Application Data	
21755	162.252801	192.168.1.66	208.103.161.1	TLSv1.2	112	Application Data	
21756	162.253780	192.168.1.66	208.103.161.1	TLSv1.2	112	Application Data	
21757	162.254853	192.168.1.66	208.103.161.1	TLSv1.2	112	Application Data	
21788	162.404475	192.168.1.66	142.250.77.106	QUIC	1292	Initial, DCID=F2c55c950715a80b, PKN: 3, PING, CRYPTO, PADDING, PING, CRYPTO, PADDING, CRYPT...	
21801	162.442671	142.250.77.106	192.168.1.66	QUIC	1292	Initial, SCID=F2c55c950715a80b, PKN: 5, CRYPTO, PADDING	
21830	163.887308	172.217.24.170	192.168.1.66	TLSv1.2	260	Application Data	
21831	163.887308	142.250.66.14	192.168.1.66	TLSv1.2	361	Application Data	
21832	163.887308	142.250.66.14	192.168.1.66	TLSv1.2	93	Application Data	
21834	163.888555	192.168.1.66	142.250.66.14	TLSv1.2	89	Application Data	
21835	163.888605	192.168.1.66	142.250.66.14	TLSv1.2	93	Application Data	
21836	163.888631	192.168.1.66	142.250.66.14	TLSv1.2	89	Application Data	
21837	163.888754	192.168.1.66	172.217.24.170	TLSv1.2	89	Application Data	
21838	163.888811	192.168.1.66	172.217.24.170	TLSv1.2	89	Application Data	
21842	163.893084	192.168.1.66	142.250.66.14	TLSv1.2	301	Application Data	
21850	164.249185	192.168.1.66	142.250.205.110	TLSv1.2	1002	Application Data	

Frame 21801: Packet, 1292 bytes on wire (10336 bits), 1292 bytes captured (10336 bits)

Ethernet II, Src: zte_9b:f9:01 (04:20:84:9b:f9:01), Dst: CloudNetwork_c2:90:15 (4c:23:00:00:00:00)

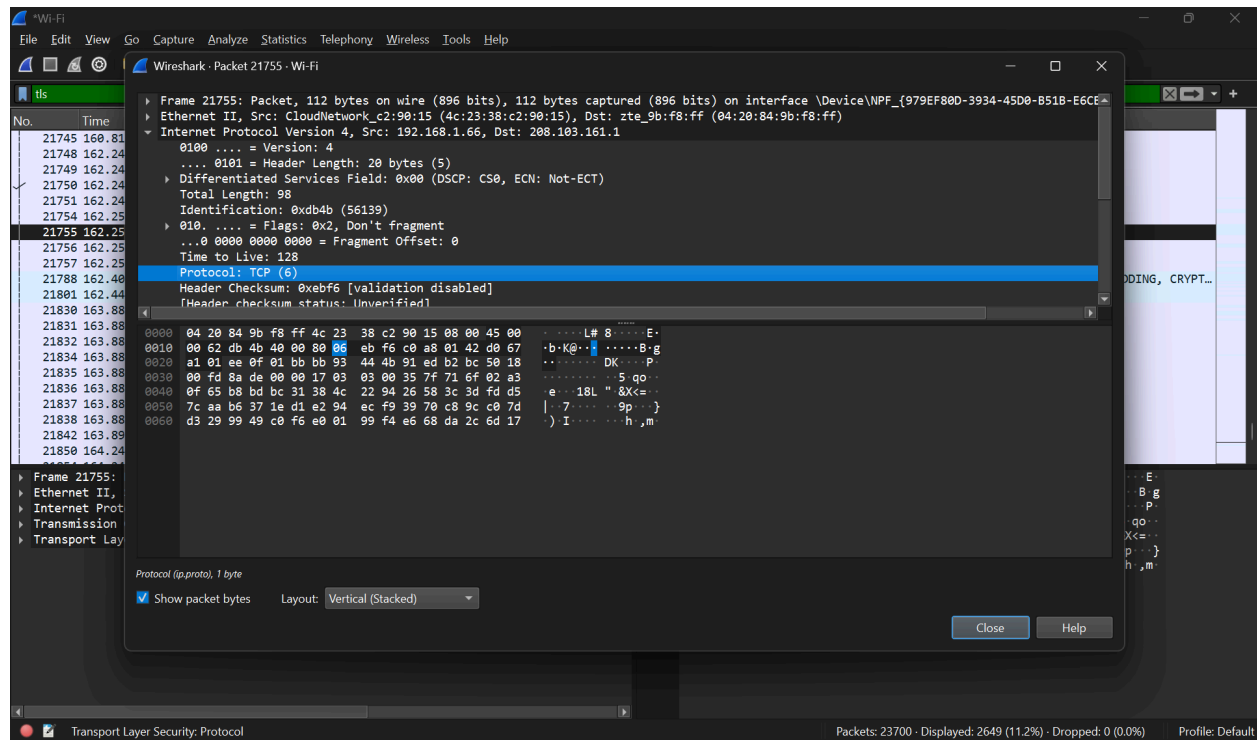
Internet Protocol Version 4, Src: 142.250.77.106, Dst: 192.168.1.66

User Datagram Protocol, Src Port: 443, Dst Port: 60676

QUIC IETF

These packets come from **secure websites like YouTube or Google**.

Step 4. Identify packet information. Click on any one packets .



This part is used for **packet analysis explanation**.

Click any packet and expand:

Internet Protocol Version 4.

5. Mobile network analysis

Mobile network analysis was performed using Wireshark by capturing packets generated from a smartphone connected to the same Wi-Fi network. After starting packet capture, activities such as browsing websites and opening mobile applications were performed on the device.

Wireshark captured packets related to the mobile device's IP address, and filters were applied to analyze the traffic. The captured packets included protocols such as DNS, TCP, TLS, and QUIC, which represent communication between the mobile device and external servers.

Step 1: Connect Mobile Device to Same Wi-Fi Network

1. Turn on **Wi-Fi on your smartphone**.
2. Connect it to the **same Wi-Fi network as your laptop/PC** running Wireshark.

This allows Wireshark to capture traffic passing through the network.

Step 2: Start Packet Capture in Wireshark

1. Open **Wireshark** on your computer.
2. Select the **Wi-Fi interface**.
3. Double-click it to **start packet capture**.

Packets will start appearing in the capture window.

No.	Time	Source	Destination	Protocol	Length	Info
5351	229.818391	192.168.1.66	172.64.155.209	TCP	54	62470 → 443 [ACK] Seq=467186 Ack=114373 Win=508 Len=0
5352	229.899321	172.64.155.209	192.168.1.66	TLsv1.2	422	Application Data
5353	229.899321	172.64.155.209	192.168.1.66	TLsv1.2	1038	Application Data
5354	229.899321	172.64.155.209	192.168.1.66	TLsv1.2	85	Application Data
5355	229.899492	192.168.1.66	172.64.155.209	TCP	54	62470 → 443 [ACK] Seq=467186 Ack=115756 Win=511 Len=0
5356	229.903822	192.168.1.66	172.64.155.209	TLsv1.2	89	Application Data
5357	229.939270	192.168.1.66	172.64.148.235	TLsv1.2	145	Application Data
5358	229.949506	172.64.155.209	192.168.1.66	TCP	60	443 → 62470 [ACK] Seq=115756 Ack=467221 Win=442 Len=0
5359	229.982923	172.64.148.235	192.168.1.66	TLsv1.2	139	Application Data
5360	230.000232	zte_9b:f8:ff	Broadcast	ARP	42	Who has 192.168.1.47? Tell 192.168.1.1
5361	230.004769	142.250.183.170	192.168.1.66	TLsv1.2	260	Application Data
5362	230.015751	192.168.1.66	142.250.183.170	TLsv1.2	89	Application Data
5363	230.015857	192.168.1.66	142.250.183.170	TLsv1.2	89	Application Data
5364	230.037707	192.168.1.66	172.64.148.235	TCP	54	61306 → 443 [ACK] Seq=1401 Ack=1429 Win=255 Len=0
5365	230.038356	142.250.183.170	192.168.1.66	TCP	54	443 → 52833 [ACK] Seq=13702 Ack=12188 Win=733 Len=0
5366	230.038356	142.250.183.170	192.168.1.66	TCP	54	443 → 52833 [ACK] Seq=13702 Ack=12223 Win=733 Len=0
5367	230.906039	192.168.1.66	172.64.155.209	TCP	55	[TCP Keep-Alive] 51447 → 443 [ACK] Seq=6006 Ack=2952 Win=255 Len=1
5368	230.924566	zte_9b:f8:ff	Broadcast	ARP	42	Who has 192.168.1.47? Tell 192.168.1.1
5369	230.928100	172.64.155.209	192.168.1.66	TCP	66	[TCP Keep-Alive ACK] 443 → 51447 [ACK] Seq=2952 Ack=6007 Win=16 Len=0 SLE=6006 SRE=6007
5370	231.602907	192.168.1.66	142.251.223.238	QUIC	1288	Protected Payload (KP0), DCID=fa18247f53e0122f

Step 3: Generate Mobile Traffic

Now create activity on your **mobile phone**.

Examples:

- Open **YouTube**
- Browse **Google**
- Open **Instagram**
- Refresh an app

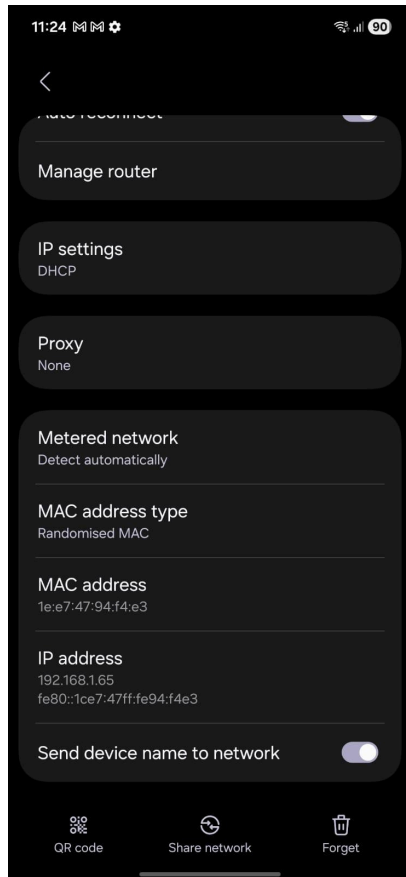
This generates network packets which Wireshark will capture.

Step 4: Identify Mobile Device Traffic

In Wireshark look for packets where the **Source or Destination IP address** matches your phone's IP.

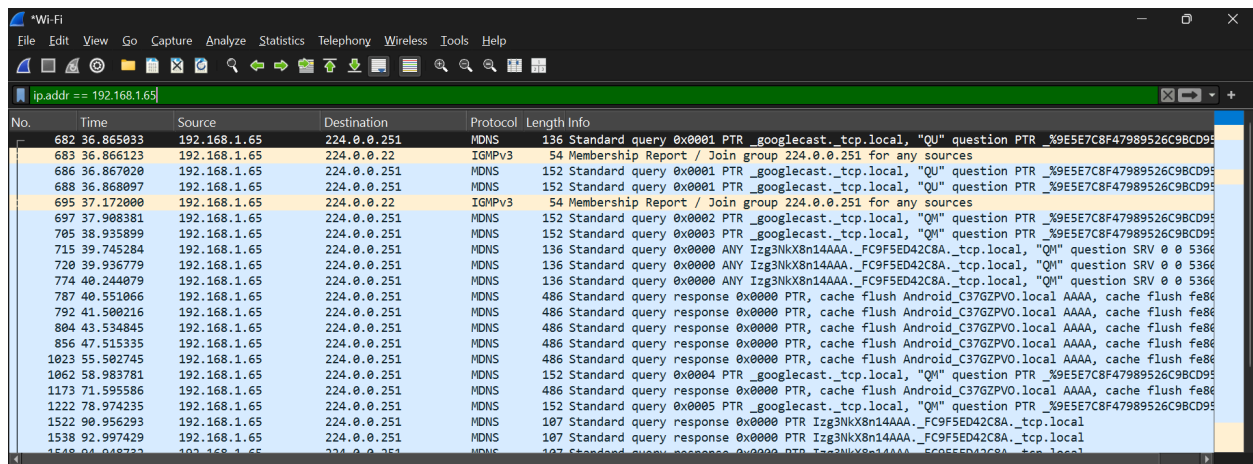
Your phone will usually have a **local IP assigned by the router**.

You can check your phone IP in: **Phone Settings** → **Wi-Fi** → **Network Details**



Step 5: Filter Mobile Traffic

To analyze only your phone's traffic, use a filter.



Now Wireshark will show **only packets related to the mobile device**.

6. IoT traffic using Wireshark

IoT traffic analysis was performed using Wireshark to monitor communication between IoT devices and network servers. Packet capture was started on the active network interface, and IoT device activity generated network traffic. Wireshark captured packets such as DNS, TCP, and HTTPS communications between the device and external cloud servers. By applying IP filters, traffic specific to the IoT device was analyzed. This analysis helps in understanding device communication behavior and identifying potential security issues in IoT networks.

Step 1: Identify IoT Device Traffic

Find the **IP address of the IoT device**.

Example format:

192.168.1.x

In Wireshark, check **Source or Destination IP** columns.

Step 2: Apply Filter for IoT Device

Use a display filter to see only traffic from the IoT device.

Example:

ip.addr == 192.168.1.30

This will display packets only from that device.

Step 3: Analyze IoT Protocols

IoT devices often use protocols such as:

- **MQTT** – messaging protocol for IoT devices
- **CoAP** – lightweight communication protocol
- **HTTP/HTTPS** – communication with cloud servers
- **DNS** – domain name resolution

Click a packet and examine details like:

- Source IP
- Destination IP
- Protocol

- Port number
- Packet length

Step 4: Observe Communication Pattern

IoT devices typically communicate with:

- **Cloud servers**
- **Mobile applications**
- **Local routers**

This communication can be observed through packet capture.